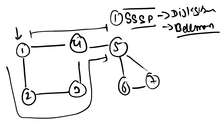
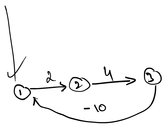
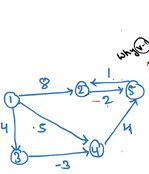
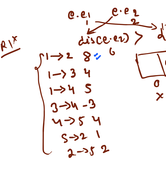
```
public void Dijkstra(int src) {
        PriorityQueue<DijkstraPair> pq = new PriorityQueue<>((a,b)->a.cost-b.cost);
        HashSet<Integer> visited = new HashSet<>();
        pq.add(new DijkstraPair(src, "" + src, 0));
        while (!pq.isEmpty()) {
            // 1. remove
            DijkstraPair rp = pq.poll();
            // 2. Ignore if ALready visited
            if (visited.contains(rp.vtx)) {
                continue;
            }
            // 3. marked visited
            visited.add(rp.vtx);
            // 4. self work
            System.out.println(rp);
            // add unvisited nbrs
            for(int nbrs:map.get(rp.vtx).keySet()) {
                if(!visited.contains(nbrs)) {
                    int cost=map.get(rp.vtx).get(nbrs);
                    pq.add(new DijkstraPair(nbrs, rp.acqpath+nbrs, rp.cost+cost));
                }
            }
        }
    }
```

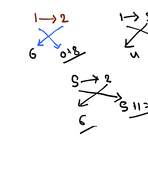


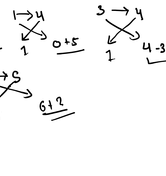




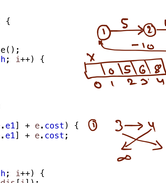




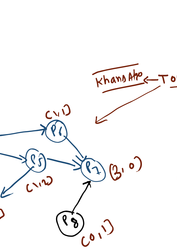




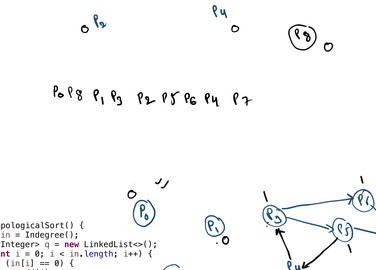


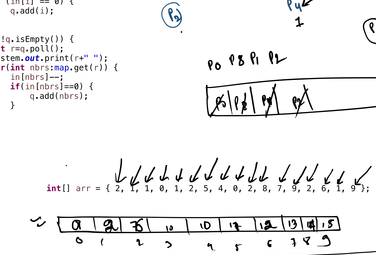


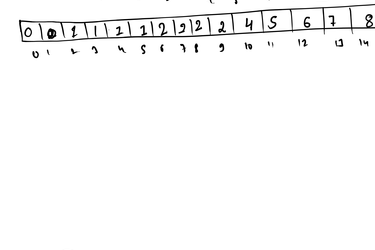
```
public void BellManFord_Algo(int src) {
        int v = map.size();
        int[] dis = new int[v + 1];
        List<EdgePair> ll = getallEdge();
        for (int i = 1; i < dis.length; i++) {
            if (i != src) {
                dis[i] = 999999;
            }
        }
        for (int i = 1; i < v; i++) {
            for (EdgePair e : ll) {
                if (dis[e.e2] > dis[e.e1] + e.cost) {
                    dis[e.e2] = dis[e.e1] + e.cost;
                }
            }
        }
        for (int i = 1; i < dis.length; i++) {
            System.out.println(i+" "+dis[i]);
        }
    }
```









```
public void TopologicalSort() {
        int[] in = Indegree();
        Queue<Integer> q = new LinkedList<>();
        for (int i = 0; i < in.length; i++) {
            if (in[i] == 0) {
                q.add(i);
            }
        }
        while(!q.isEmpty()) {
            int r=q.poll();
            System.out.print(r+" ");
            for(int nbrs:map.get(r)) {
                in[nbrs]--;
                if(in[nbrs]==0) {
                    q.add(nbrs);
                }
            }
        }
    }
```